

R의 객체지향 프로그래밍

R 프로그래밍 with workflow design



R의 객체지향 프로그래밍 기초

객체 지향 프로그래밍

- **객체 지향 프로그래밍 (**Objective-Oriented Programming: OOP**)**
  - 프로그램을 **객체(**object**)** 중심으로 구성하는 패러다임
  - 객체 구성요소
    - **상태(**state**)**: 객체의 속성(변수)
    - **행동(**behavior**)**: 객체가 수행할 수 있는 동작으로 메서드(함수)로 정의됨
- R에서는 모든 데이터를 “객체”라고 부르지만 이 장에서는 OOP **관점의 객체(**object**)**와 R **데이터 오브젝트(**object**)**를 구분해서 사용한다.

R에서의 객체지향 프로그래밍과 S3 시스템

- R도 OOP를 지원하며 초급 사용자도 무의식 중에 OOP 구조를 사용하고 있음
- 하지만 통합된 OOP 시스템은 없고 필요에 따라 S3, S4, R5, R6 시스템 중 선택적으로 사용
- R base 및 대부분의 CRAN 패키지가 S3 시스템 기반이므로 이 장에서는 가장 널리 사용되는 S3 시스템에 집중
- **S3 시스템의 특징**
  - 속성과 클래스를 자유롭게 정의 가능 (구조가 느슨함)
  - 객체 내부에 메서드가 있는 구조가 아닌 제너릭 함수가 외부에서 메서드를 선택
  - 예: `print()` 와 `summary()` 등은 제너릭 함수로 클래스에 따라 다른 메서드가 작동
- 이 장에서는 제너릭 함수가 작동하는 원리를 중심으로 OOP 구조를 이해할 예정

클래스와 속성

S3 시스템에서의 클래스 정의

- S3 시스템은 포멀한 클래스 선언 없이 단순히 **클래스 속성(class attribute)**을 새롭게 정의하고 사용하는 방식
- `class()`

```
num_vec = 1:5
class(num_vec) = "new_class"
print(num_vec) ; class(num_vec)
```

```
## [1] 1 2 3 4 5
## attr(,"class")
## [1] "new_class"
```

```
## [1] "new_class"
```

- `structure()`: 오브젝트 생성과 동시에 클래스 속성 부여

```
num_vec2 = structure(1:5, class = "new_class")
print(num_vec2)
```

```
## [1] 1 2 3 4 5
## attr(,"class")
## [1] "new_class"
```

data.frame의 클래스와 속성 구조 이해

```r
df = data.frame(a = 1:4, b = 5:8) # 예시 데이터프레임 생성
class(df) # 객체의 클래스 확인
```

```
## [1] "data.frame"
```

```r
attributes(df) # 객체의 모든 속성 (attributes) 출력
```

```
## $names
## [1] "a" "b"
##
## $class
## [1] "data.frame"
##
## $row.names
## [1] 1 2 3 4
```

```r
names(df) # 열 이름만 따로 확인
```

```
## [1] "a" "b"
```

제너릭 함수

제너릭 함수(Generic Function)란?

- **제너릭 함수(Generic Function)**: 객체의 클래스에 따라 다르게 작동하는 함수
  - 대표적 예시: `print()`, `plot()`, `summary()`
- 하나의 함수 이름으로 다양한 클래스의 객체에 대해 다르게 작동하는 방식을 구현
  - 예: print()는 벡터, 행렬, 데이터프레임, 모형 결과 등 각 클래스에 맞는 출력 수행

print() 함수 내부

```
print
```

```
## function (x, ...)
## UseMethod("print")
## <bytecode: 0x00000146b4da2890>
## <environment: namespace:base>
```

- UseMethod()는 제너릭 함수임을 선언하는 방식
- 클래스가 lm이면 print.lm(), data.frame이면 print.data.frame()이 실행

S3 시스템에서 메서드 호출 구조

```r
x = lm(mpg ~ wt, data = mtcars)
print(x)   # 내부적으로는 print.lm(x)가 호출됨
```

```
##
## Call:
## lm(formula = mpg ~ wt, data = mtcars)
##
## Coefficients:
## (Intercept)           wt
##      37.285       -5.344
```

- 사용자는 `print(x)` 만 입력하지만 R은 `class(x)` 를 확인하고 적절한 메서드 함수를 자동 선택

어떤 메서드가 존재하는지 확인

```
methods("print")[1:6]
```

```
## [1] "print.acf"                    "print.activeConcordance"
## [3] "print.AES"                    "print.anova"
## [5] "print.aov"                    "print.aovlist"
```

- `methods("함수이름")`을 통해 해당 제너릭 함수에 정의된 모든 클래스별 메서드 목록 확인 가능
- `print.data.frame()`, `print.lm()` 등으로 직접 접근도 가능

제너릭 함수와 메서드의 구현

제너릭 함수 정의 예시

```r
my_function = function(x) UseMethod("my_function")

# 기본 메서드
my_function.default = function(x) {
  cat("일반 객체입니다:", x, "\n")
}

# person_info 클래스 전용 메서드
my_function.person_info = function(x) {
  cat("이름:", x$name, "나이:", x$age, "\n")
}
```

- 메서드는 함수명.클래스명 형식으로 작성
- S3 시스템의 규칙을 따름

객체 생성 및 적용 예

```
p = list(name = "Kwan-Young Bak", age = 35)
class(p) = "person_info"

my_function(42)   # 일반 객체

## 일반 객체입니다: 42
my_function(p)    # person_info 객체

## 이름: Kwan-Young Bak 나이: 35
```

예제: 스플라인 기저 함수 플랏 메서드

S3 객체로 스플라인 기저 함수 시각화

- 목적: `create_design_matrix()` 결과에 클래스를 부여하고, `plot()` 제너릭 함수에 맞는 S3 메서드를 구현해보기
- 이점
  - 분석 결과를 객체처럼 다룰 수 있음
  - `plot()` 호출만으로 자동으로 기저 함수 시각화 가능

[복습] b_spline() 함수

```r
b_spline = function(x, knots, degree, i) {
  # 0차 스플라인 기저함수의 경우
  if (degree == 0) {
    return(ifelse(knots[i] <= x & x < knots[i + 1], 1, 0))
  }

  # 재귀적인 경우
  B_i_d1 = b_spline(x, knots, degree - 1, i)
  B_i1_d1 = b_spline(x, knots, degree - 1, i + 1)
  denom1 = knots[i + degree] - knots[i]
  denom2 = knots[i + degree + 1] - knots[i + 1]
  term1 = if (denom1 == 0) 0
          else ((x - knots[i]) / denom1) * B_i_d1
  term2 = if (denom2 == 0) 0
          else ((knots[i + degree + 1] - x) / denom2) * B_i1_d1
  return(term1 + term2)
}
```

설계 행렬 클래스 지정

```
create_design_matrix = function(x_values, knots, degree){
  n = length(x_values)
  num_basis = length(knots) - degree - 1
  design_matrix = matrix(0, nrow = n, ncol= num_basis)

  for(j in 1:num_basis)
    for(i in 1:n)
      design_matrix[i,j] = b_spline(x_values[i], knots, degree, j)

  # 속성 부여 추가
  attr(design_matrix, "x_values") = x_values # 벡터 x값 저장
  attr(design_matrix, "knots") =knots # 매듭점 시퀀스 저장
  attr(design_matrix, "degree") = degree # 스플라인 차수 저장
  class(design_matrix) ="basis" # "basis" 클래스 속성 부여

  return(design_matrix)
}
```

plot.basis() 메서드 정의

```r
plot.basis = function(basis_obj, ...) {
  x_values = attr(basis_obj, "x_values")
  if (is.null(x_values))
    stop("x_values attribute is missing in the basis object.")

  matplot(x_values, basis_obj, type = "l", lty = 1,
          col = rainbow(ncol(basis_obj)),
          xlab = "x", ylab = "B-spline value",...)
}
```

- `plot()` 함수 호출 시 클래스 "basis"를 감지하여 이 함수 실행
- `matplot()`을 활용해 여러 개의 기저 함수를 한 번에 선 그래프로 출력
- `...`는 사용자로부터 전달된 추가 그래픽 옵션을 `matplot()`에 그대로 넘겨주는 역할

사용 예시

```r
degree = 3
tiny = 1e-5
knots = c(rep(0- tiny, degree), 0, 1, 2, 3, 4, rep(4 + tiny, degree
x_values = seq(0, 4, length.out = 200)

basis_obj = create_design_matrix(x_values, knots, degree)
attributes(basis_obj)$knots
```

```
##  [1] -0.00001 -0.00001 -0.00001  0.00000  1.00000  2.00000  3.00
##  [9]  4.00001  4.00001  4.00001
```

```r
class(basis_obj)
```

```
## [1] "basis"
```

S3 메서드를 이용한 시각화

```
plot(basis_obj)
```

